

The Vedak Hash Function Family

Yu Zhang, Yuhan Zhang, Shanpeng Wei, Si Gao, Wenling Wu

Institute of Software, Chinese Academy of Sciences, Beijing 100190, China

Zhongguancun Laboratory, Beijing, China

2026.06.30

Abstract

This document specifies the Vedak hash function family and presents its design rationale, security analysis, and implementation performance. Vedak is based on the sponge construction and a large-state permutation. Vedak supports digest lengths of 512, 768, and 1024 bits, corresponding to Vedak-512, Vedak-768, and Vedak-1024, respectively.

The underlying permutation of Vedak is the large-state cryptographic permutation Vedak-p, which is suitable for SIMD instruction sets. Vedak uses nibbles as the basic processing unit, employs 4-bit S-boxes, nibble slicing, shuffle, and other operations. Vedak fully considers the functionality of AVX2, AVX-512, and NEON instruction sets, and its software performance is superior to SHA3. At the same time, nibble slicing adopts the optimal binary field matrix, using S-box properties together with branch number arguments, providing provable security against typical analysis methods. Vedak has good scalability, adopts the sponge construction, uses a large-state cryptographic permutation to support high-security-strength parameter choices. Vedak is easily extensible to support multiple digest lengths, suitable for application requirements with multiple security strengths. At the same time, Vedak inherits the advantages and mature security analysis conclusions of the sponge construction, resists length extension attacks. Under the ideal-permutation model for the sponge construction, Vedak inherits the corresponding generic security bounds against classical and quantum attacks.

Contents

1	Introduction	4
2	The Specification of Vedak	4
2.1	Notation	4
2.2	Overview	4
2.3	Message Padding	6
2.4	Vedak-p	6
2.4.1	ν	8
2.4.2	λ	8
2.4.3	ψ	9
2.4.4	ω	10
3	Design Rationale	10
3.1	Overall Structure	10
3.2	Vedak-p	11
3.2.1	ν	12

3.2.2	λ	12
3.2.3	ψ	13
3.2.4	ω	17
4	Security Analysis	17
4.1	Differential Cryptanalysis	18
4.1.1	MILP-aided Framework Based on Generative Constraint Modeling	18
4.1.2	Differential Cryptanalysis of Vedak-p	23
4.2	Linear Cryptanalysis	25
4.3	Meet-in-the-Middle Attack	26
5	Software Performance	28
5.1	Implementation Performance using AVX2 Instruction Set	29
5.2	Implementation Performance using AVX-512 Instruction Set	32
6	Features	33
7	Statistical Tests	34
7.1	Statistical Randomness Test Results of Vedak-512	34
7.2	Statistical Randomness Test Results of Vedak-768	36
7.3	Statistical Randomness Test Results of Vedak-1024	36
8	Instance	36
8.1	Vedak-512	36
8.2	Vedak-768	39
8.3	Vedak-1024	41

1 Introduction

In response to the demand for high-security cryptographic hash function in the collection of new-generation commercial cryptographic algorithms, we have designed Vedak. We provide algorithm description, design principles, security analysis, and implementation performance evaluation. The design goal of Vedak is a high-security, high-performance hash function. Its main technical targets follow the requirements of the call for submissions and are summarized as follows:

1. The algorithm simultaneously supports three digest lengths: 512, 768, and 1024 bits;
2. The algorithm resists known classical and quantum attacks;
3. The algorithm can be efficiently implemented on a wide range of software and hardware platforms and is practical for deployment;

2 The Specification of Vedak

2.1 Notation

The following notation is used throughout the description of Vedak:

Symbol	Explanation
$\oplus$	bitwise XOR
$\parallel$	bit-string concatenation
$\lll m$	left cyclic shift by m bits
$\lll_{32} m$	blockwise 32-bit cyclic shift by m bits
$\{0, 1\}^n$	n -bit vector, $\{0, 1\}^*$ represents vectors of arbitrary bit length
$ X $	the length of the bit vector X
0^n	n -bit all-zero vector
$X[i : j]$	the $i, i + 1, \dots, j$ bits of X
mod	modulo operation
$\equiv$	identity

2.2 Overview

Vedak is a family of cryptographic hash functions based on the Sponge construction[1], as shown in Fig. 1. Vedak supports three digest output lengths, namely 512, 768, and 1024 bits, and the corresponding variants are denoted by Vedak-512, Vedak-768, and Vedak-1024, respectively. The specific parameters are shown in Table 1. For any input message $M \in \{0, 1\}^*$, Vedak outputs a l -bit digest Z . The complete procedure is specified in Algorithm 1.

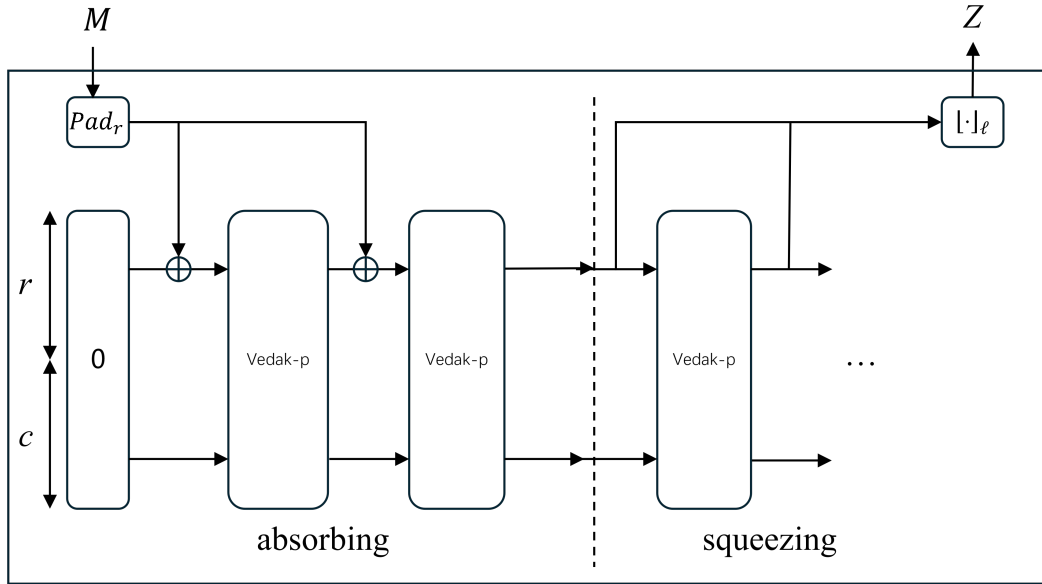


Figure 1: The overall structure of Vedak

Table 1: Vedak algorithm parameters

Variant	Digest length ℓ	Rate r	Capacity c
Vedak-512	512	1536	1024
Vedak-768	768	1024	1536
Vedak-1024	1024	512	2048

Algorithm 1 Vedak hash function

Require: rate r , message $M \in \{0, 1\}^*$, output length $\ell > 0$

Ensure: digest $Z \in \{0, 1\}^\ell$

```
1:  $M' \leftarrow \text{pad}_r(M)$  ▷ message padding
2:  $n \leftarrow |M'|/r$  ▷ message partitioning
3: Parse  $M'$  as  $M_0 \| M_1 \| \dots \| M_{n-1}$ , where  $M_i \in \{0, 1\}^r$ 
4:  $S \leftarrow 0^{2560}$  ▷ initialization
5: for  $i = 0$  to  $n - 1$  do
6:    $S \leftarrow S \oplus (M_i \| 0^{2560-r})$  ▷ absorb one message block
7:    $S \leftarrow \text{Vedak-p}(S)$  ▷ invoke Vedak-p
8: end for
9:  $Z^* \leftarrow S[0 : r - 1]$ 
10: while  $|Z^*| < \ell$  do
11:    $S \leftarrow \text{Vedak-p}(S)$  ▷ continue squeezing
12:    $Z^* \leftarrow Z^* \| S[0 : r - 1]$ 
13: end while
14:  $Z \leftarrow Z^*[0 : \ell - 1]$ 
15: return  $Z$ 
```

2.3 Message Padding

Vedak adopts the multi-rate padding rule pad_{10^*1} . Concretely, a bit 1, followed by as few bits 0 as possible, and then another bit 1 are appended to the right of the message M , so that the length of the resulting padded bit string becomes a positive multiple of r . Formally,

$$\text{pad}_r(M) = M \| 1 \| 0^t \| 1, \quad t = (-|M| - 2) \bmod r. \quad (1)$$

Hence, the padded message

$$M' = \text{pad}_r(M) \quad (2)$$

satisfies $|M'| \equiv 0 \pmod{r}$. This padding rule guarantees a unique mapping from every input message to an r -aligned bit string.

2.4 Vedak-p

Vedak-p is an iterative permutation with 2560 bits and 40 iterations. Vedak-p uses 4-bit nibbles as the basic unit. The 2560-bit internal state can be regarded as 640 nibbles, or as 40 64-bit words, or as 80 32-bit words. Let $X = (X_0, X_1, \dots, X_{39})$, $X_t = (x_{t,0}, x_{t,1}, \dots, x_{t,15}) \in (F_2^4)^{16}$,

$t = 0, 1, \dots, 39$, denotes an internal state. We represent the state of Vedak-p using a 10×4 matrix array, as shown in Fig. 2.

$$\begin{pmatrix} X_0 & X_1 & X_2 & X_3 \\ X_4 & X_5 & X_6 & X_7 \\ \vdots & \vdots & \vdots & \vdots \\ X_{36} & X_{37} & X_{38} & X_{39} \end{pmatrix}$$

Figure 2: Vedak-p internal state

For ease of description, the state of Vedak-p can also be represented in the two-dimensional form shown in Fig. 3, satisfying $X_t = X_{i,j}, t = 4i + j, t = 0, 1, \dots, 39$.

$$\begin{pmatrix} X_{0,0} & X_{0,1} & X_{0,2} & X_{0,3} \\ X_{1,0} & X_{1,1} & X_{1,2} & X_{1,3} \\ \vdots & \vdots & \vdots & \vdots \\ X_{9,0} & X_{9,1} & X_{9,2} & X_{9,3} \end{pmatrix}$$

Figure 3: Two-dimensional representation of Vedak-p internal state

The round function of Vedak-p consists of the following four steps: ν , λ , ψ , and ω . The details are specified in Algorithm 2.

Algorithm 2 Vedak-p

Input: $X \in (F_2^4)^{640}$.

Output: $Y \in (F_2^4)^{640}$.

- 1: **for** $Round = 1$ to 40 **do**
 - 2: $U \leftarrow \nu(X)$.
 - 3: $V \leftarrow \lambda(U)$.
 - 4: $W \leftarrow \psi(V)$.
 - 5: $X \leftarrow \omega(W)$.
 - 6: **end for**
 - 7: $Y \leftarrow X$.
 - 8: **return** Y .
-

2.4.1 ν

Apply 16 4-bit S-boxes in parallel to each $X_t = (x_{t,0}, x_{t,1}, \dots, x_{t,15})$ ($t = 0, 1, \dots, 39$),

$$U_t \leftarrow (s(x_{t,0}), s(x_{t,1}), \dots, s(x_{t,15})), t = 0, 1, \dots, 39.$$

The S-box of Vedak-p is a 4-bit to 4-bit bijection $s : \{0, 1\}^4 \rightarrow \{0, 1\}^4$, defined by the following lookup table.

x	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
$s(x)$	1	0	5	6	C	9	2	8	A	7	3	F	E	B	4	D

2.4.2 λ

Apply the 10×10 matrix $\mathbf{M}$ in parallel to 4 columns of the internal state, shown in Fig. 4.

$$\begin{array}{cccc}
 \begin{pmatrix} U_{0,0} \\ U_{1,0} \\ \vdots \\ U_{9,0} \end{pmatrix} & \begin{pmatrix} U_{0,1} \\ U_{1,1} \\ \vdots \\ U_{9,1} \end{pmatrix} & \begin{pmatrix} U_{0,2} \\ U_{1,2} \\ \vdots \\ U_{9,2} \end{pmatrix} & \begin{pmatrix} U_{0,3} \\ U_{1,3} \\ \vdots \\ U_{9,3} \end{pmatrix} \\
 \downarrow \mathbf{M} & \downarrow \mathbf{M} & \downarrow \mathbf{M} & \downarrow \mathbf{M} \\
 \begin{pmatrix} V_{0,0} \\ V_{1,0} \\ \vdots \\ V_{9,0} \end{pmatrix} & \begin{pmatrix} V_{0,1} \\ V_{1,1} \\ \vdots \\ V_{9,1} \end{pmatrix} & \begin{pmatrix} V_{0,2} \\ V_{1,2} \\ \vdots \\ V_{9,2} \end{pmatrix} & \begin{pmatrix} V_{0,3} \\ V_{1,3} \\ \vdots \\ V_{9,3} \end{pmatrix}
 \end{array}$$

Figure 4: λ operation

Applying the λ operation to the internal state U yields V ,

$$\begin{pmatrix} V_{0,j} \\ V_{1,j} \\ \dots \\ V_{9,j} \end{pmatrix} \leftarrow \mathbf{M} \begin{pmatrix} U_{0,j} \\ U_{1,j} \\ \dots \\ U_{9,j} \end{pmatrix}, j = 0, 1, 2, 3.$$

The details of the matrix $\mathbf{M}$ are as follows:

$$\mathbf{M} = \begin{bmatrix} 1 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 1 \end{bmatrix}.$$

2.4.3 ψ

Apply the nibble-oriented permutation $P^{(i)}$ ($i = 0, 1, \dots, 9$) to every row of the internal state. The internal states V, W can be represented as the 80 32-bit words shown in Fig. 5, satisfying $V_{i,j} = \tilde{V}_{i,2j} \parallel \tilde{V}_{i,2j+1}, W_{i,j} = \tilde{W}_{i,2j} \parallel \tilde{W}_{i,2j+1}, i = 0, 1, \dots, 9, j = 0, 1, 2, 3$.

$$\begin{pmatrix} \tilde{V}_{0,0} & \tilde{V}_{0,1} & \cdots & \tilde{V}_{0,7} \\ \tilde{V}_{1,0} & \tilde{V}_{1,1} & \cdots & \tilde{V}_{1,7} \\ \vdots & \vdots & \ddots & \vdots \\ \tilde{V}_{9,0} & \tilde{V}_{9,1} & \cdots & \tilde{V}_{9,7} \end{pmatrix} \begin{pmatrix} \tilde{W}_{0,0} & \tilde{W}_{0,1} & \cdots & \tilde{W}_{0,7} \\ \tilde{W}_{1,0} & \tilde{W}_{1,1} & \cdots & \tilde{W}_{1,7} \\ \vdots & \vdots & \ddots & \vdots \\ \tilde{W}_{9,0} & \tilde{W}_{9,1} & \cdots & \tilde{W}_{9,7} \end{pmatrix}$$

Figure 5: The 80 words representation of V, W

ψ is an operation oriented to 32-bit words, applying different nibble-oriented permutation $P^{(i)}$ to each row, as illustrated in Fig. 6.

$$\begin{array}{ccc} \left[\tilde{V}_{0,0} & \tilde{V}_{0,1} & \cdots & \tilde{V}_{0,7} \right] & \xrightarrow{\mathbf{P}^{(0)}} & \left[\tilde{W}_{0,0} & \tilde{W}_{0,1} & \cdots & \tilde{W}_{0,7} \right] \\ \left[\tilde{V}_{1,0} & \tilde{V}_{1,1} & \cdots & \tilde{V}_{1,7} \right] & \xrightarrow{\mathbf{P}^{(1)}} & \left[\tilde{W}_{1,0} & \tilde{W}_{1,1} & \cdots & \tilde{W}_{1,7} \right] \\ \vdots & & \vdots & & & \vdots \\ \left[\tilde{V}_{9,0} & \tilde{V}_{9,1} & \cdots & \tilde{V}_{9,7} \right] & \xrightarrow{\mathbf{P}^{(9)}} & \left[\tilde{W}_{9,0} & \tilde{W}_{9,1} & \cdots & \tilde{W}_{9,7} \right] \end{array}$$

Figure 6: ψ operation

$P^{(i)}$ consists of a permutation $P_w^{(i)}$ targeting 32-bit words and a blockwise 32-bit cyclic shift $\lll_{32} m_i$. The specific values of $P_w^{(i)}$ and m_i are shown in Table 2.

Table 2: The value of $P_w^{(i)}$ and m_i

i	$P_w^{(i)}$	m_i
0	(5, 4, 7, 6, 1, 0, 3, 2)	16
1	(2, 3, 0, 1, 6, 7, 4, 5)	4
2	(6, 7, 4, 5, 2, 3, 0, 1)	12
3	(7, 6, 5, 4, 3, 2, 1, 0)	0
4	(3, 2, 1, 0, 7, 6, 5, 4)	28
5	(0, 1, 2, 3, 4, 5, 6, 7)	0
6	(0, 1, 3, 2, 4, 5, 7, 6)	20
7	(4, 5, 7, 6, 0, 1, 3, 2)	12
8	(5, 4, 6, 7, 1, 0, 2, 3)	20
9	(1, 0, 2, 3, 5, 4, 6, 7)	4

2.4.4 ω

Inject the round constant at the first 8 nibbles. Let $RC_d \in \{0, 1\}^{32}$ be the 32-bit round constant for the d th round, and the specific values of RC_d are shown in Table 3.

Table 3: Vedak-p round constant

$RC_0 = 2c387d69$	$RC_1 = 988cc9dd$	$RC_2 = f0e4a1b5$	$RC_3 = 21357064$
$RC_4 = 8397d2c6$	$RC_5 = c7d39682$	$RC_6 = 4f5b1e0a$	$RC_7 = 5e4a0f1b$
$RC_8 = 7c682d39$	$RC_9 = 392d687c$	$RC_{10} = b3a7e2f6$	$RC_{11} = a7b3f6e2$
$RC_{12} = 8e9adfc b$	$RC_{13} = dcc88d99$	$RC_{14} = 786c293d$	$RC_{15} = 30246175$
$RC_{16} = a1b5f0e4$	$RC_{17} = 8296d3c7$	$RC_{18} = c5d19480$	$RC_{19} = 4a5e1b0f$
$RC_{20} = 55410410$	$RC_{21} = 6b7f3a2e$	$RC_{22} = 17034652$	$RC_{23} = effbbeaa$
$RC_{24} = 1f0b4e5a$	$RC_{25} = ffebaeba$	$RC_{26} = 3f2b6e7a$	$RC_{27} = bfabeefa$
$RC_{28} = beaaeffb$	$RC_{29} = bca8edf9$	$RC_{30} = b9ade8fc$	$RC_{31} = b2a6e3f7$
$RC_{32} = a5b1f4e0$	$RC_{33} = 8b9fdace$	$RC_{34} = d7c38692$	$RC_{35} = 6f7b3e2a$
$RC_{36} = 1e0a4f5b$	$RC_{37} = fde9acb8$	$RC_{38} = 3a2e6b7f$	$RC_{39} = b4a0e5f1$

3 Design Rationale

3.1 Overall Structure

Vedak follows the standard Sponge construction and splits the internal state into a rate part and a capacity part. During absorption, message blocks are injected only into the rate part, while the capacity part participates in the state update only through the underlying permutation;

during squeezing, output is read from the rate part, and when more than one rate block is needed the underlying permutation is applied again and squeezing continues.

Vedak adopts an internal state of size $b = 2560$ bits. This parameter choice supports a 1024-bit long digest while still leaving relatively high rates for Vedak-512 and Vedak-768, thus achieving a balance between security and implementation efficiency.

3.2 Vedak-p

Vedak-p is the underlying permutation of the Vedak. The design of Vedak-p is well suited for SIMD instruction sets. Using the vector permutation instructions provided by AVX2/AVX-512, the nonlinear 4-bit S-box layer can be implemented efficiently. What's more, implementing the linear layer in nibble-slice form only requires XOR operations and vector permutation instructions, making the software implementation efficient. Fig. 7 shows the nibble-slice structure of Vedak-p. Linear transformation is oriented towards 32-bit words and is ARM-friendly. Due to use the SP structure, adopting relevant conclusions of S-boxes and branch number, Vedak-p has provable security bounds against differential cryptanalysis and linear cryptanalysis. Furthermore, relatively mature theoretical frameworks for cryptanalysis and security evaluation have been established for impossible differential cryptanalysis, meet-in-the-middle attacks. Vedak-p has good diffusion property, and full diffusion is achieved after 5 rounds.

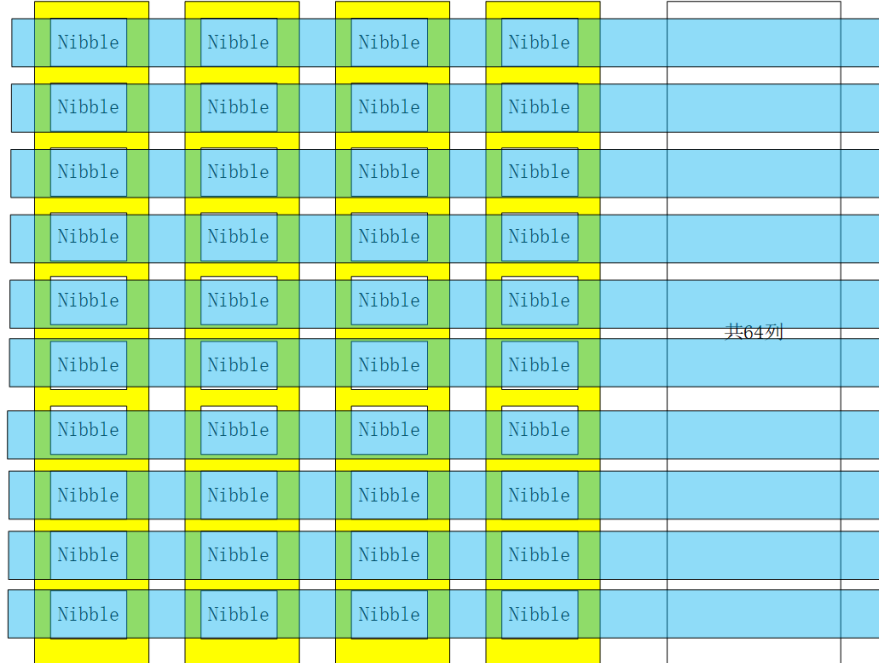


Figure 7: Nibble-slice structure of Vedak-p

3.2.1 ν

The choice of a 4-bit S-box in Vedak-p is that 4-bit S-boxes are highly suitable for fast software implementation using SIMD instruction sets.

The S-box adopted by Vedak has no fixed points and has the optimal differential uniformity 4, nonlinearity 4, algebraic degree 3, algebraic degree 3 of every component function. Moreover, it does not allow any difference Δ to be mapped to the same Δ with a high probability, say 2^{-2} for a 4-bit S-box. Furthermore, this S-box has the shortest maximum high-probability chain length 2 [23].

3.2.2 λ

The design of λ is based on the following three aspects. Firstly, the implementation cost is dominated by XOR instructions in the nibble-sliced representation. Secondly, the processing can be done in parallel using AVX2/AVX-512 instruction sets. Thirdly, λ is based on the optimal binary matrix of 10×10 , achieving large differential and linear branch numbers at low XOR cost.

The overall structure of λ is shown in Fig. 8. Using AVX2 instruction set and storing each row of the Vedak-p internal state(as shown in Fig. 3) in two 256-bit registers, it only takes $5 \times 5 \times 2$ XOR instructions to perform the λ operation on 4 columns in parallel. What's more, the cost of rotation and permutation is free. Using AVX-512 instruction set, compared to AVX2 instruction set, the implementation cost is reduced by half. Storing each row of the Vedak-p internal state in one 512-bit register, it only takes 5×5 XOR instructions to complete the λ operation on 4 columns in parallel. Referring the idea of constructing optimal diffusion components from iterative Feistel structures[3], the 10×10 matrix adopted in λ operation reaches both differential and linear branch numbers equal to 6, which is optimal in binary domain.

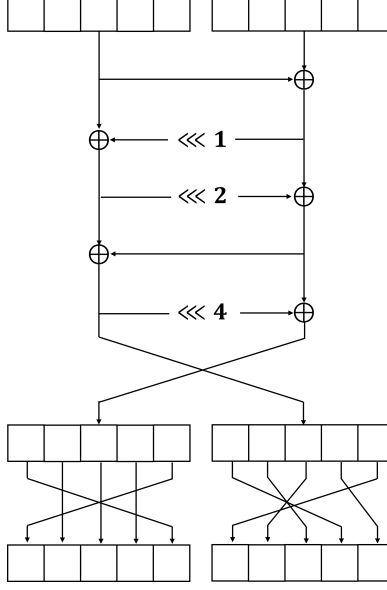


Figure 8: The overall structure of λ

3.2.3 ψ

The design of ψ is based on the following two aspects. Firstly, ψ is friendly to SIMD instruction set. Secondly, ψ has good diffusion property with the lowest possible implementation cost. ψ operation is used to further disrupt the correlations between different positions in the internal state after λ operation, achieving full diffusion in as few rounds as possible. ψ operation is composed of permutation $P_w^{(i)}$, oriented to 32-bit words, and blockwise 32-bit cyclic shift, natively compatible with ARM architecture. Furthermore, $P_w^{(i)}$ is composed of permutation oriented to 64-bit words and 32-bit block swap within a 64-bit word.

Permutation oriented to 64-bit words Reorder the four 64-bit words in each row i ($i = 0, 1, \dots, 9$):

$$(X_{i,0}, X_{i,1}, X_{i,2}, X_{i,3}) \xrightarrow{P_i} (X_{i,P_i[0]}, X_{i,P_i[1]}, X_{i,P_i[2]}, X_{i,P_i[3]}), \quad i = 0, 1, \dots, 9.$$

Considering the software implementation performance and adaptability, permutation oriented to 64-bit words should be chosen from the set

$$\{[0, 1, 2, 3], [2, 3, 0, 1]\},$$

as much as possible. Using the AVX2 instruction set, the implementation cost of this type of permutation is zero. The specific information of P_i is listed in Table 4.

Table 4: The value of P_i

i	0	1	2	3	4
P_i	[2,3,0,1]	[1,0,3,2]	[3,2,1,0]	[3,2,1,0]	[1,0,3,2]
i	5	6	7	8	9
P_i	[0,1,2,3]	[0,1,2,3]	[2,3,0,1]	[2,3,0,1]	[0,1,2,3]

32-bit block swap. Divide the 64-bit word into two 32-bit block:

$$x = x_L \parallel x_R, \quad x_L, x_R \in F_2^{32}.$$

$$Swap_{32}(x) = x_R \parallel x_L.$$

If the swap flag $\alpha_{i,j} = 1, (i = 0, 1, \dots, 9, j = 0, 1, 2, 3)$ in position (i, j) , perform the 32-bit block swap operation on the element at this position; otherwise, leave it unchanged. Let X and Y be the input and output respectively, the following relationship is satisfied.

$$Y_{i,j} = \begin{cases} Swap_{32}(X_{i,j}), & \alpha_{i,j} = 1, \\ X_{i,j}, & \alpha_{i,j} = 0. \end{cases}$$

Using AVX2 instruction set and storing each row in two 256-bit registers, it only takes 2 shuffle instructions to complete the 32-bit block swap operation on all 64-bit word for each row. Using AVX-512 instruction set, the cost is reduced by half. Storing each row in one 512-bit registers, it only takes 1 shuffle instructions. To enhance the implementation performance, the number of 32-bit block swap operations should be minimized on the premise of ensuring diffusion and security. The detailed value of $\alpha_{i,j}$ is illustrated in Table 5.

Table 5: The value of $\alpha_{i,j}$

i	$\alpha_{i,0}$	$\alpha_{i,1}$	$\alpha_{i,2}$	$\alpha_{i,3}$
0	1	1	1	1
1	0	0	0	0
2	0	0	0	0
3	1	1	1	1
4	1	1	1	1
5	0	0	0	0
6	0	1	0	1
7	0	1	0	1
8	1	0	1	0
9	1	0	1	0

In the process of specific parameter screening, optimal full diffusion serves as one of the criteria. The linear layer composed of λ and above ψ candidate structure achieves full diffusion after 5 rounds. The specific reasons are as follows.

The internal state of Vedak-p can be regarded as 64 parallel slices and each slice corresponding to a 10-dimensional linear layer. From the perspective of diffusion activity, we exhaustively enumerate the propagation relationships of 2^{10} active patterns within a single slice. The following results are obtained. For one round, if a slice of the output achieves all 10 components to become active after one round, the input must have at least 3 active blocks. In other words, to achieve 4-round full diffusion, there must be at least 3 active blocks in the input of the 4th round. There are 64 parallel slices in total. Therefore, to achieve 4-round full diffusion, the number of active blocks in the input of the 4th round must be at least $3 \times 64 = 192$. However, it can be only propagated to 5 active blocks when the number of input active blocks is minimum. Furthermore, combining the fact that the number of active blocks can be magnified by 6 times at most, the number of active blocks of the 4th round input is no more than $5 \times 6 \times 6 = 180$. Therefore, full diffusion cannot be achieved in 4 rounds. Based on this conclusion, we have included 5-round full diffusion as one of the screening criteria. What's more, the diffusion in 3 rounds and 4 rounds, implementation cost, and security are comprehensively evaluated to determine ψ . The specification is as follows:

1. Search 5-round full diffusion candidate. It is infeasible to traverse all space of 640-length permutations. Therefore, we adopt a hierarchical search strategy, searching step by step at three different granularities, i.e, 40-length permutations $\rightarrow$ 80-length permutations $\rightarrow$ 640-length permutations. The specification is as follows.

(1) 40-length (oriented to 64-bit words) permutation screening: Layout the internal

state X in a 10×4 format (as shown in Fig. 3). Search for implementation-friendly 40-length permutations. Specifically:

- (i) Apply different 4-length permutations to each row: 4-length permutations are oriented to 64-bit words and are friendly with 64-bit software platforms (such as x86-64).
- (ii) Limiting the first two 64-bit words and the last two 64-bit words of each row to always be in the same half-region, that is, selecting the 4-length permutation from the set

$$\{[0, 1, 2, 3][0, 1, 3, 2], [1, 0, 2, 3], [1, 0, 3, 2], [2, 3, 0, 1], [2, 3, 1, 0], [3, 2, 0, 1], [3, 2, 1, 0]\},$$

avoids cross-register movement in AVX2 and has low implementation cost. Specifically, the implementation cost of permutations $[0, 1, 2, 3]$ and $[2, 3, 0, 1]$ is zero, while permutations $[0, 1, 3, 2]$ $[1, 0, 2, 3]$ $[2, 3, 1, 0]$ and $[3, 2, 0, 1]$ require only one Permute instruction, and permutations $[1, 0, 3, 2]$ and $[3, 2, 1, 0]$ require only two Permute instructions.

- (iii) The total number of swaps between two 128-bit lanes is limited to 8. In AVX2's 256-bit registers, swapping two 128-bit lanes requires a Permute instruction. Considering implementation performance, it is advisable to minimize the number of swaps between two 128-bit lanes.
- (iv) Leave at least one row unchanged to reduce the number of instructions.

Under the aforementioned constraints: Firstly, candidates (oriented to 64-bit words) that can achieve 3-round full diffusion in both forward and backward directions are selected. Then, the sum of the number of diffusion blocks in 2 rounds is used as the sorting metric.

(2) 80-length (oriented to 32-bit words) permutation screening: To balance portability for low-end implementations (such as ARM), each 64-bit word is split into two 32-bit words, with the intermediate states arranged in a 10×8 layout (as shown in Fig. 5). This step only considers whether to swap the expansion degrees of freedom of the left and right 32-bit words, and ensures a balanced permutation distribution. Candidates (oriented to 32-bit words) that meet the criteria of both forward and backward full diffusion rounds being 4 rounds are filtered. Then, further screening is conducted using the number of diffusion blocks in the forward 3 rounds as an indicator.

(2) 640-length (oriented to nibble) permutation screening: Each 32-bit word is regarded as 8 nibbles. A unified nibble cyclic right shift amount is introduced for each row, and each shift amount is limited to appearing at most twice in 10 rows. Candidates satisfying both forward and backward full diffusion rounds of 5 rounds (nibble-oriented) are screened. Then, further screening was conducted using the number of forward 4-round diffusion blocks as an indicator.

2. From diffusion to security. Candidate permutations are indistinguishable in terms of implementation cost/diffusion property, thus further criteria need to be introduced. We evaluate the differential and linear security of each candidate permutation and select the one with the best security within our computational capacity.

3.2.4 ω

To distinguish rounds and break potential symmetric structures, Vedak-p injects a 32-bit round constant in each round. Referring to [5], the round constant is generated by an 8-stage linear feedback shift register (LFSR). This method of constant generation has a simple structure, low hardware cost, and is easy to implement.

4 Security Analysis

Vedak adopts the Sponge construction. Its top-level security is primarily determined by the capacity parameter c and the security of the underlying public permutation. Under the ideal permutation model, the Vedak-p permutation is regarded as a uniformly random permutation. Vedak inherits the universal security bounds of the Sponge construction in the sense of a random oracle [1, 6, 7]. The security claims are presented in Table 6.

Table 6: The security claim of Vedak

Variant	r	c	ℓ	Collision security	Preimage / second-preimage security
Vedak-512	1536	1024	512	2^{256}	2^{512}
Vedak-768	1024	1536	768	2^{384}	2^{768}
Vedak-1024	512	2048	1024	2^{512}	2^{1024}

Vedak follows the Sponge construction. Therefore, Vedak is naturally resistant to length-extension attacks. In the quantum setting, the security of Vedak likewise inherits the corresponding results for the Sponge construction in the model of quantum-accessible oracles[10]. Combining the generic results on Grover search and quantum collision search[11, 12, 13], Vedak-512, Vedak-768, and Vedak-1024 provide quantum preimage security levels of 256, 384, and 512 bits, respectively. Their quantum collision security is jointly constrained by the output length and the capacity, and cannot exceed the complexity bound achieved by generic quantum collision attacks for the corresponding output length.

In summary, the top-level security of Vedak is mainly derived from the generic bounds of the Sponge construction, while the concrete security further depends on the ability of Vedak-p to

withstand structural cryptanalysis. We first evaluate the security of the permutation Vedak-p.

Since Vedak-p is the core public permutation of Vedak, its structural security margin directly determines the security of concrete Vedak instantiations. Using automated cryptanalysis methods, we evaluate the security of Vedak-p from the perspective of classical cryptanalysis, focusing on its resistance to differential, linear attacks. The results indicate that Vedak-p has sufficient security margin.

4.1 Differential Cryptanalysis

Differential cryptanalysis is a classic method for evaluating the security of ciphers. However, for large-state permutations like Vedak-p, existing analysis techniques cannot return effective solutions within a reasonable time. To address this issue, we propose a MILP-aided framework tailored for large-state primitives. This framework is based on generative constraint modeling. With the new search method, a lower bound of the number of differential active S-boxes for Vedak-p is obtained. There are at least 630 differential active S-boxes in 30 rounds of Vedak-p.

4.1.1 MILP-aided Framework Based on Generative Constraint Modeling

The λ transformation of Vedak-p provides diffusion with a differential branch number of 6 on each column of 10 nibbles, while the ψ transformation further ensures that active nibbles in the same column are distributed to different columns in subsequent rounds, thus satisfying typical wide-trail strategy [15]. Based on this, a lower bound can be directly obtained. Vedak-p contains at least 36 differential active S-boxes per 4 rounds. However, when the number of rounds increases to 5 or more, it becomes difficult to obtain sufficiently tight results solely relying on the branch number method. Meanwhile, for large-state permutations like Vedak-p, directly modeling its feasible differential propagation would lead to a rapid expansion of the number of constraints, making it difficult to solve effectively. To address this, we adopt a generative constraint modeling approach, modeling differential propagation patterns through linear layer with a set of compact constraints, thereby significantly improving the scalability of solving while maintaining modeling accuracy.

When modeling the differential propagation of complex linear layers, precise modeling can lead to poor solving performance, while modeling only based on differential branch number faces accuracy bottlenecks. To alleviate this contradiction, we propose Generative Constraint Modeling (GCM). The basic idea is to bypass explicit modeling of XOR relationships and instead use the constraint construction criterion of "not excluding feasible solution sets". By generating a constraint family with strong exclusion ability, we can reduce infeasible propagation paths, thereby improving solving efficiency.

Constraint and solution space. Automatic modeling can be uniformly described as a constraint solving problem. For boolean variable set $\mathcal{V}$ and constraint set $\mathcal{C}$, define the feasible solution space as,

$$\mathcal{S}(\mathcal{C}) := \bigcap_{C \in \mathcal{C}} \mathcal{S}(C),$$

where $\mathcal{S}(C)$ represent the feasible solution space for single constraint C .

When $\mathcal{S}(\mathcal{C}) = \emptyset$, the constraint system is unsatisfiable. Given the objective function to minimize f , we obtain a constrained optimization problem, whose optimal value is:

$$f^* = \min_{x \in \mathcal{S}(\mathcal{C})} f(x).$$

The core issue we focus on is: constructing a set of "sufficiently strong" constraints to eliminate infeasible propagation as much as possible, without excluding genuinely feasible solutions.

Unit-Cover Inequality. The basic components of GCM is a special class of linear inequalities,

$$\sum_{i \in I} x_i + \sum_{j \in J} (1 - x_j) \geq 1, \quad (3)$$

where $x_i \in \{0, 1\}$ I, J are disjoint sets.

This linear inequality is equal to a CNF clause,

$$\left(\bigvee_{i \in I} x_i \vee \bigvee_{j \in J} \neg x_j \right),$$

That is, if the CNF formula is true, at least one literal is true.

We refer to equation (3) as the Unit-Cover Inequality (UCI). An important characteristic of this constraint is that it precisely excludes a locally fixed boolean sub-cube, thus providing a clear geometric interpretation and a low decision cost.

Definition 1 (The length and strength of UCI). *Let $\mathcal{X} = \{x_1, \dots, x_n\}$ be boolean variable set. For the unit-cover inequality (3), let variable set corresponding to it be $I \cup J$ and let k be the length of this constraint, where*

$$k := |I \cup J|.$$

Define the strength of constraint as

$$\text{strength} = n - k. \quad (4)$$

The strength can be seen as the logarithmic scale of the number of boolean assignments that can be excluded by the constraint. When only k variables are fixed with specific values, the

remaining $n - k$ variables can take on any value freely. Therefore, the infeasible assignment scale that is excluded is of the same order as 2^{n-k} . In the generative constraint modeling, we tend to generate UCI with higher strength (i.e., shorter length) to achieve more effective search space reduction.

The key to generative constraints lies not in writing a "complete and precise" static constraint set at once, but in constructing a set of constraint variants that satisfy the implication relationship.

Definition 2 (The solution space implication relationship of constraint variants). *Let original constraint model be $\mathcal{M} = (\mathcal{V}, \mathcal{C})$, where $\mathcal{V}$ be variable set and $\mathcal{C}$ be constraint set. For another constraint set $\mathcal{C}'$*

- *If $\mathcal{S}(\mathcal{C}) \subseteq \mathcal{S}(\mathcal{C}')$, then $\mathcal{C}'$ is called loosen constraint variant of $\mathcal{C}$.*
- *If $\mathcal{S}(\mathcal{C}') \subseteq \mathcal{S}(\mathcal{C})$, then $\mathcal{C}'$ is called strong constraint variant of $\mathcal{C}$.*

If it is an optimization problem $\min f(x)$, then the optimal value corresponding to the loosen variant gives a lower bound. The optimal value corresponding to the strong variant provides an upper bound. For lower bound solving, like minimum number of active S boxes, constructing appropriate loosen variants to enhance solving efficiency can still yield meaningful security lower bounds.

Consider that there are m S boxes on the input of the linear layer, and their active states are represented by $\mathbf{a} \in \{0, 1\}^m$, The active state on the output is represented by $\mathbf{b} \in \{0, 1\}^m$. Combine the two,

$$\mathbf{x} = (\mathbf{a}, \mathbf{b}) \in \{0, 1\}^{2m}.$$

Let $\mathcal{F} \subseteq \{0, 1\}^{2m}$ be the set of all feasible solution. It can be computed by Algorithm 3. $\overline{\mathcal{F}}$ is infeasible set. The goal of generative constraints is to construct the constraint family $\mathcal{C}_{\text{gen}}$ and make

$$\mathcal{F} \subseteq \mathcal{S}(\mathcal{C}_{\text{gen}}), \tag{5}$$

That is, the generated constraints will not exclude any feasible propagation, while excluding as many points in $\overline{\mathcal{F}}$ as possible.

Definition 3 (Effective Unit-Cover Inequality). *For boolean variable set $\mathcal{X} = \{x_1, \dots, x_{2m}\}$, if unit-cover inequality C satisfies:*

$$\mathcal{F} \subseteq \mathcal{S}(C),$$

then C is called effective unit-cover inequality.

Algorithm 3 Feasibility test for active S-box pattern propagation through a linear layer

- d : dimension of the linear layer
 $M \in \mathbb{F}_2^{d \times d}$: linear layer
 X, Y : two d -dimensional Boolean vectors
- 1: **Step 1 (Initialization & Collection).**
 - 2: Form a $2d$ -length array P such that $P[0..d-1] \leftarrow X$ and $P[d..2d-1] \leftarrow Y$. For each row i of M , collect the indices j where $M_{i,j} = 1$ and $P[j] = 1$, storing them in $OC[i]$, and record the count in $OI[i]$. If $P[i+d] = 1$ but $OI[i] < 1$, or $P[i+d] = 0$ but $OI[i] = 1$, **return** false (infeasible).
 - 3: **Step 2 (Construct $mseq$).**
 - 4: Build a matrix $mseq$ by selecting rows i with $P[i+d] = 0$ and marking $mseq[i, OC[i][.]] = 1$. Perform row operations (e.g., Gaussian elimination) on $mseq$ to obtain its rank mo . If any row in $mseq$ has fewer than two active bits, **return** false.
 - 5: **Step 3 (Construct $fseq$).**
 - 6: Similarly, form a matrix $fseq$ by selecting rows i with $P[i+d] = 1$ and marking $fseq[i, OC[i][.]] = 1$. Use the pivot rows from $mseq$ to reduce $fseq$. If any row in $fseq$ becomes a zero row after reduction, **return** false.
 - 7: **Step 4 (Conclusion).**
 - 8: If all checks pass, **return** true, indicating that $X \rightarrow Y$ is a feasible active S-box propagation pattern under matrix M .
-

Here is a basic fact: any infeasible point can correspond to a UCI that excludes only that point.

Theorem 1. *For any $\mathbf{x} \in \overline{\mathcal{F}}$, there exists a $2m$ -length unit-cover inequality $C_{\mathbf{x}}$, only excluding $\mathbf{x}$ and satisfying $\mathcal{F} \subseteq \mathcal{S}(C_{\mathbf{x}})$.*

Proof. Let $\mathbf{x} = (x_1, \dots, x_{2m}) \in \{0, 1\}^{2m}$. Construct

$$C_{\mathbf{x}} : \sum_{i:x_i=1} (1 - x_i) + \sum_{i:x_i=0} x_i \geq 1,$$

Clearly, the left side is 0 only when the assignment is equal to $\mathbf{x}$. For all other assignments, the left side is at least 1. Therefore, this constraint only excludes $\mathbf{x}$. Meanwhile, since $\mathbf{x} \notin \mathcal{F}$, it follows that $\mathcal{F} \subseteq \mathcal{S}(C_{\mathbf{x}})$ holds. $\square$

Theorem 1 presents a class of benchmark constraints with strength 0. To obtain stronger pruning capability, it is necessary to reduce the length from $2m$ to enhance the strength (see Eq. (4)). To this end, a greedy pruning generation strategy is adopted. Starting from $C_{\mathbf{x}}$, it attempts to gradually remove variables from the constraints. As long as the propagation validity is still satisfied after deletion, the deletion operation should be retained. The output constraints of algorithm 4 are typically shorter than $C_{\mathbf{x}}$, thus exhibiting higher constraint strength. At the implementation level, the pruning decision that $\mathcal{F} \subseteq \mathcal{S}(C')$ can be accomplished through an enumeration check of $\mathcal{F}$ or a sampling check of its subsets.

Algorithm 4 Greedy-UCI-Generator

Input: Feasible solution set $\mathcal{F} \subseteq \{0, 1\}^{2m}$, infeasible solution set $\overline{\mathcal{F}}$, expected number of inequalities N
Output: Effective unit-cover inequality $\mathcal{C}$

```
1:  $\mathcal{C} \leftarrow \emptyset$ 
2: while  $|\mathcal{C}| < N$  do
3:   Select one seed  $\mathbf{x} = (x_1, \dots, x_{2m})$  from  $\overline{\mathcal{F}}$ .
4:   Construct initial constraint  $C \leftarrow C_{\mathbf{x}}$ 
5:   for  $i = 1$  to  $2m$  do
6:     Let  $C'$  be the candidate constraint obtained by deleting the  $i$ th variable from  $C$ .
7:     if  $\mathcal{F} \subseteq \mathcal{S}(C')$  then
8:        $C \leftarrow C'$  ▷ Pruning successful.
9:     end if
10:  end for
11:   $\mathcal{C} \leftarrow \mathcal{C} \cup \{C\}$ 
12: end while
13: return  $\mathcal{C}$ 
```

To further enhance the pruning effect and generation efficiency of constraints, we adopt the following two strategies.

Firstly, seed selection biased towards low-weight infeasible points. During the propagation of the difference through linear layer, infeasible points are denser in the low-weight region, making it easier to generate strong constraints starting from low Hamming weight $\mathbf{x} \in \overline{\mathcal{F}}$.

Secondly, a threshold T is introduced to reduce the overhead of determining propagation validity. When m is large, directly verifying that $\mathcal{F} \subseteq \mathcal{S}(C')$ over the entire set $\mathcal{F}$ incurs high computational cost. At this point, only feasible points satisfying $W(\mathbf{x}) \leq T$ can be checked in the decision phase, For high-weight areas, point-by-point verification is not performed, in exchange for higher generation efficiency.

Under the threshold strategy, an auxiliary boolean variable s is introduced to construct the following generalized form,

$$\sum_{i \in I} x_i + \sum_{j \in J} (1 - x_j) + s \geq 1. \quad (6)$$

And add the bounding constraint

$$W(\mathbf{x}) \geq (T + 1) \cdot s. \quad (7)$$

When $W(\mathbf{x}) > T$, $s = 1$ is allowed, so that equation (6) is automatically satisfied; When $W(\mathbf{x}) \leq T$, s must be set to 0, resulting in the constraint degenerating to the standard UCI, which continues to exert pruning effects on low-weight regions.

Generative constraint modeling takes the feasible set $\mathcal{F}$ as the semantic benchmark, and achieves batch elimination of infeasible propagation by constructing a UCI family. Compared to explicit XOR modeling, this method emphasizes directly reducing the search space with a small

number of high-strength constraints, thus providing flexible modeling and solving capabilities in complex linear layers and large-state cryptanalysis scenarios.

4.1.2 Differential Cryptanalysis of Vedak-p

Let the input/output active patterns of a single 10-dimensional linear layer be denoted as binary vectors $\mathbf{x}, \mathbf{y} \in \{0, 1\}^{10}$, where a component of 1 indicates that the corresponding nibble is active. Combine ($\mathbf{x}$ and $\mathbf{y}$) into a 20-dimensional 0-1 vector $\mathbf{v} = (\mathbf{x} \parallel \mathbf{y}) \in \{0, 1\}^{20}$. Based on Algorithm 3, all pattern pairs were enumerated and their feasibility was determined. The results showed that a total of 927139 differential active propagation patterns were infeasible, while 121437 were feasible. We use all infeasible propagation patterns as seeds and apply Algorithm 4 to prune, iterate, and remove duplicates to obtain a set of logical constraints $\mathcal{C}$ with a size of 4160, which satisfies:

- Any infeasible propagation $\mathbf{v}$ must violate at least one constraint $C \in \mathcal{C}$.
- Any feasible propagation is not excluded by $\mathcal{C}$.

Thus, the differential propagation feasible region of the linear layer can be characterized equivalently in MILP using $\mathcal{C}$.

Greedy selection of generative constraints. Since Vedak-p contains 64 parallel linear layers in each round, directly using all 4160 constraints will significantly increase the burden of solving. To address this, we adopt the greedy strategy proposed by Sun et al. [22]: at each step, select the constraint that can cover the largest number of remaining infeasible propagations, and remove its covered part from the infeasible set. Repeat this process until a balance is achieved between the coverage effect and the constraint scale. Table 7 provides statistics for some constraints in this process, where L is the clause length, Δ is the number of new coverages added in this round, Σ is the cumulative number of coverages, R is the number of remaining infeasible solutions, and C is the selected clause). It can be observed that only using the first 70 constraints can cover about two-thirds of infeasible propagation. Considering both the coverage effect and solving efficiency, select the first 70 constraints as the generative constraint set to model the feasible differential propagation in each linear layer of Vedak-p.

Let the objective function be the minimum number of active super S-boxes in 6 rounds of Vedak-p, and add the aforementioned generative constraints to MILP model. On the AMD EPYC 9754 128-Core Processor platform, using no more than 400 threads, it took approximately 4 days to obtain that Vedak-p contains at least 21 differential active super S-boxes per 6 rounds. The solving process is shown in Figure 9.

Table 7: Greedy selection of generative constraints

No.	L	Δ	Σ	R	Clause
1	6	16384	16384	910755	$(x_3 \vee \neg x_5 \vee x_6 \vee x_9 \vee x_{10} \vee x_{17})$
2	6	16384	32768	894371	$(x_1 \vee x_5 \vee x_7 \vee x_{12} \vee x_{13} \vee \neg x_{17})$
3	6	16384	49152	877987	$(x_2 \vee x_5 \vee x_9 \vee x_{11} \vee \neg x_{13} \vee x_{20})$
4	6	16384	65536	861603	$(\neg x_9 \vee x_{12} \vee x_{15} \vee x_{16} \vee x_{17} \vee x_{20})$
5	6	16384	81920	845219	$(x_1 \vee x_5 \vee x_7 \vee \neg x_{12} \vee x_{13} \vee x_{17})$
6	6	15872	97792	829347	$(\neg x_1 \vee x_2 \vee x_8 \vee x_{13} \vee x_{14} \vee x_{18})$
7	6	15360	113152	813987	$(x_1 \vee x_5 \vee \neg x_7 \vee x_{12} \vee x_{13} \vee x_{17})$
8	6	15360	128512	798627	$(x_1 \vee \neg x_5 \vee x_9 \vee x_{16} \vee x_{18} \vee x_{19})$
9	6	15104	143616	783523	$(x_4 \vee x_5 \vee x_{10} \vee \neg x_{13} \vee x_{14} \vee x_{19})$
10	6	14400	158016	769123	$(\neg x_1 \vee x_5 \vee x_7 \vee x_{12} \vee x_{13} \vee x_{17})$
...					
70	6	3694	585672	341467	$(x_2 \vee x_4 \vee x_7 \vee \neg x_{12} \vee x_{14} \vee x_{20})$
...					
4159	10	1	927138	1	$(x_3 \vee x_5 \vee \neg x_6 \vee x_8 \vee x_9 \vee x_{13} \vee x_{14} \vee x_{18} \vee x_{19} \vee x_{20})$
4160	10	1	927139	0	$(x_1 \vee x_2 \vee x_6 \vee x_8 \vee x_{10} \vee \neg x_{11} \vee x_{12} \vee x_{16} \vee x_{18} \vee x_{19})$

Since each super active S-box corresponds to 6 active S-boxes, it can be further deduced that

$$N_6^{\text{diff}} \geq 21 \times 6 = 126,$$

where N_6^{diff} denotes the minimum number of differential active S-boxes in 6 rounds of Vedak-p. Furthermore, for any 30-round differential characteristic, the minimum number of active S-boxes is

$$N_{30}^{\text{diff}} \geq 5 \times 126 = 630.$$

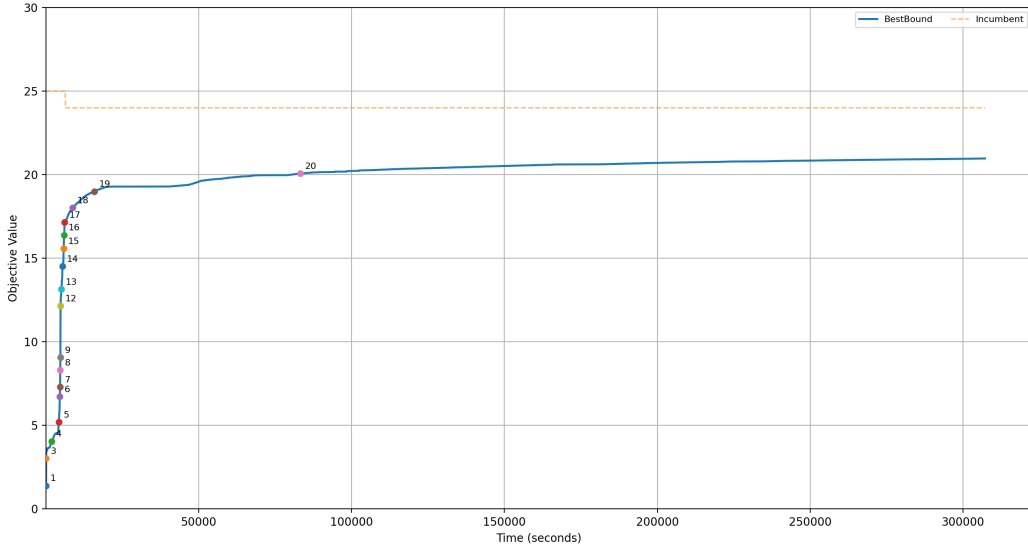


Figure 9: Search process of the minimum differential super active S-box for 6-round Vedak-p

4.2 Linear Cryptanalysis

Linear cryptanalysis is a classical method for evaluating the security of ciphers, but for large-state permutations like Vedak-p, existing analysis techniques cannot return an effective solution within a reasonable time. To address this issue, we search a lower bound of the minimum number of linear active S-boxes for Vedak-p based on a GCM-based MILP search framework. The search results indicate that there are at least 630 linear active S-boxes for 30-round Vedak-p.

Linear cryptanalysis [18] and differential cryptanalysis share a high degree of similarity in their modeling structures, making it possible to evaluate the former within a framework akin to that of differential analysis. A fundamental lower bound can be initially obtained through the wide-trail strategy [15]. There are at least 36 linear active S-boxes for 4-round Vedak-p. When the number of rounds increases to 5 rounds and above, it is also difficult to provide a sufficiently tight security bound only relying on branch number. Similar to differential cryptanalysis, we employ the GCM framework to search the number of linear active S-boxes. The results show that there are at least 21 linear super S-boxes for 6-round Vedak and the detailed search process is shown in Fig. 10. Each active super S-box corresponds to 6 active S-boxes. Therefore,

$$N_6^{\text{lin}} \geq 21 \times 6 = 126,$$

Furthermore,

$$N_{30}^{\text{lin}} \geq 5 \times 126 = 630.$$

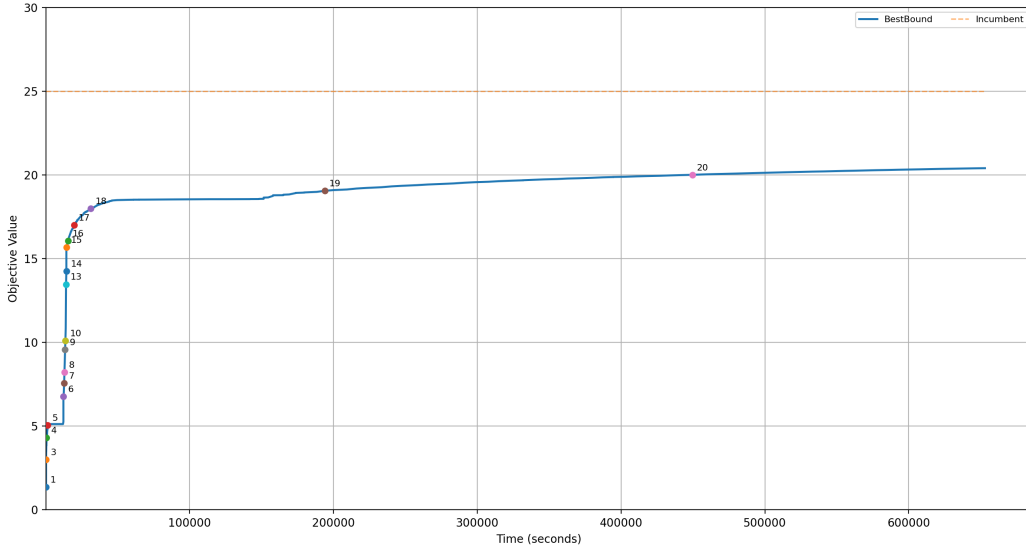


Figure 10: Search process of the minimum linear super active S-box for 6-round Vedak-p

4.3 Meet-in-the-Middle Attack

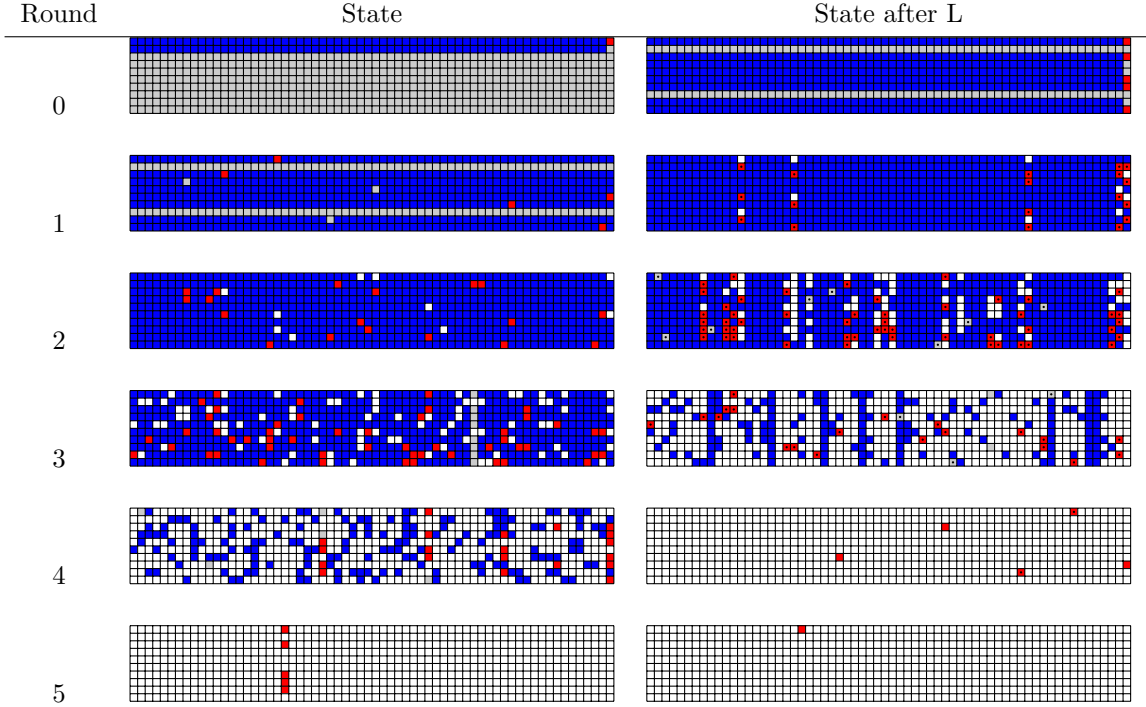


Figure 11: Meet-in-the-middle characteristic for reduced 5-round Vedak

We evaluate the resistance of reduced-round Vedak to meet-in-the-middle attacks in the sponge setting. Fig. 11 shows a 5-round MITM characteristic for the underlying permutation. The characteristic is described at the nibble-cell level. The blue, red, gray, and white cells follow the standard dependency separation used in sponge MITM attacks, and the cells marked with a central dot indicate the consumption of blue degrees of freedom [24, 25].

The characteristic applies to preimage attacks on all three 5-round *Vedak-n* variants, and it also gives a collision attack on *Vedak-1024*. The resulting time and memory complexities are summarized below.

Table 8: Table: Complexities of the MITM attacks on reduced 5-round Vedak

Attack type	Variant	Time complexity	Memory complexity
Preimage attack	Vedak-512	2^{508}	2^{504}
Preimage attack	Vedak-768	2^{764}	2^{504}
Preimage attack	Vedak-1024	2^{1020}	2^{504}
Collision attack	Vedak -1024	2^{510}	2^{510}

The preimage attack procedure on *Vedak*-1024 is given in Algorithm 5. The attack uses a three-block message structure: the first two message blocks are used to tune the internal state before the 5-round MITM interval; the third block is determined by a pair of blue and red variables that passes the partial matching condition, and the resulting candidate is finally verified against the full target digest.

Algorithm 5 Three-block preimage attack on reduced 5-round *Vedak*-1024

Require: Digest length n , target digest Z

Ensure: A message M such that $H(M) = Z$

```

1: for  $2^{n-508}$  independent choices of the first two blocks  $(M_0, M_1)$  do
2:   Compute the internal state before entering the 5-round MITM interval.
3:   Build the blue table  $\mathcal{U}$ : enumerate  $2^{504}$  blue assignments and store them according to the
      consumed value  $C_B$ .
4:   for all required values of  $C_B$  do
5:     for all assignments  $r$  to the red cell do
6:       Perform the one-cell blue–red partial matching test.
7:       for each matched blue assignment  $b \in \mathcal{U}[C_B]$  do
8:         Construct the third message block  $M_2$  from  $(b, r)$ .
9:         if  $M = M_0 \| M_1 \| M_2$  satisfies  $H(M) = Z$  then
10:           return  $M$ .
11:         end if
12:       end for
13:     end for
14:   end for
15: end for
16: return failure.

```

The collision attack procedure on *Vedak*-1024 is given in Algorithm 6. The attack is obtained by converting the partial-target preimage attack into a birthday collision search, and only two message blocks are needed.

Algorithm 6 Collision attack on reduced 5-round *Vedak*-1024

Require: Selected output cell T , target value $\tau \in \text{GF}(2^4)$

Ensure: Two distinct messages $M \neq M'$ such that $H(M) = H(M')$

```
1: Initialize the global digest table  $\mathcal{H} \leftarrow \emptyset$ .
2: for  $2^6$  independent choices of the first block  $M_0$  do
3:   Compute the internal state after absorbing  $M_0$ .
4:   Build the blue table  $\mathcal{U}$ : enumerate  $2^{504}$  blue assignments and store them according to the
      consumed value  $C_B$ .
5:   for all required values of  $C_B$  do
6:     for all assignments  $r$  to the red cell do
7:       Perform the one-cell blue–red partial matching test.
8:       for each matched blue assignment  $b \in \mathcal{U}[C_B]$  do
9:         Construct  $M_1$  from  $(b, r)$ , and set  $M = M_0 \| M_1$ .
10:        Compute  $Z \leftarrow H(M)$ .
11:        if  $Z[T] = \tau$  then
12:          if  $\mathcal{H}[Z]$  already stores a message  $M' \neq M$  then
13:            return  $(M, M')$ .
14:          else
15:            Store  $M$  in  $\mathcal{H}[Z]$ .
16:          end if
17:        end if
18:      end for
19:    end for
20:  end for
21: end for
22: return failure.
```

Since the full-diffusion round number of Vedak is 5, even a conservative round estimate based on the meet-in-the-middle resistance criterion for MD-type constructions, which usually leads to a larger number of attackable rounds than in the sponge setting, does not yield a MITM characteristic beyond 20 rounds. Therefore, the full 40-round Vedak algorithm provides a sufficient diffusion margin and is expected to resist meet-in-the-middle-type attacks.

5 Software Performance

Vedak fully utilizes the nibble slicing layout and SIMD instruction set, enabling the S-box layer, column diffusion layer, and row transformation layer to be completed with fewer instructions. The main operations in the underlying permutation only involve 4-bit S-boxes, XORs, vector permutations, etc., making it particularly suitable for implementation on modern CPUs with

AVX2, AVX-512, and ARM NEON instruction sets.

5.1 Implementation Performance using AVX2 Instruction Set

We evaluate the software performance of Vedak on different PCs. The results show that Vedak-512 outperforms SHA3-512.

Performance on AMD EPYC 9754. The first testing environment consisted of an AMD EPYC 9754 128-core CPU running Ubuntu 24.04, with g++ 13.3.0 and OpenSSL 3.0.13. Table 9 presents the speed test results for various message lengths.

To facilitate horizontal comparison, all three Vedak instances use SHA3-512 as the baseline for comparison. The performance of software implementation is given in cycles per byte (cpb) The test results indicate that, with the AVX2 instruction set, Vedak-512 outperforms SHA3-512 in terms of performance across short, medium, and long messages, with particularly superior performance in long message testing. Compared to SHA3-512, Vedak-512 offers a speed increase of approximately 43.3%. Furthermore, Vedak-768 is also faster than SHA3-512.

Table 9: Software performance of Vedak on AMD EPYC 9754 (cycle/Byte)

Data Level	Message length	SHA3-512	Vedak-512	Vedak-768	Vedak-1024
S1	32B	46.406	34.438	33.031	64.688
S2	128B	17.164	8.609	16.344	32.867
S3	512B	12.539	6.328	10.238	20.785
S4	1024B	11.110	6.262	9.272	18.765
S5	4096B	10.027	5.708	8.498	17.260
S6	8192B	9.883	5.576	8.377	17.010
S7	16384B	9.826	5.574	8.315	16.886
S8	65536B	9.787	5.544	8.276	16.802

Performance on AMD Ryzen 7. The second test environment is Windows 10, The processor is an AMD Ryzen 7 5800X 8-Core Processor running at 3.80 GHz, Memory: 16.0 GB. The compilation environment is g++, with compilation options `-std=c++20 -mavx2 -march=native`. SHA3-512 is implemented using OpenSSL’s `libcrypto` library for speed measurement. In this test environment, compared to SHA3-512, Vedak-512 offers a speed increase of approximately 29.5%. Table 10 and Table 11 presents the speed test results for various message lengths.

Table 10: Software performance of Vedak on AMD Ryzen 7 (cycle/Byte)

Message length	SHA3-512	Vedak-512	Vedak-768	Vedak-1024
8B	223.250	199.500	213.750	437.000
32B	55.812	51.062	53.438	108.062
64B	27.906	25.531	26.719	81.344
68B	26.265	24.029	25.147	77.676
80B	31.825	20.425	21.375	66.025
96B	26.125	17.021	17.812	55.417
100B	25.460	16.340	17.480	52.820
128B	19.891	13.062	25.234	55.516
200B	16.340	15.960	16.530	43.320
300B	16.087	10.640	16.213	34.960
400B	13.965	11.875	16.150	35.245
576B	13.722	10.951	13.920	33.712
1536B	11.553	9.228	13.434	29.984
4096B	10.817	8.461	12.756	28.203
> 4096B (1MiB)	10.717	8.278	12.330	27.565

Table 11: Software performance of Vedak on AMD Ryzen 7 (Mbps)

Message length	SHA3-512	Vedak-512	Vedak-768	Vedak-1024
8B	136.17	152.38	142.22	69.57
32B	544.69	595.35	568.88	281.32
64B	1089.37	1190.71	1137.77	373.72
68B	1157.43	1265.14	1208.89	391.37
80B	955.22	1488.37	1422.22	460.43
96B	1163.64	1786.03	1706.71	548.57
100B	1194.03	1860.47	1739.13	575.54
128B	1528.33	2327.36	1204.72	547.59
200B	1860.47	1904.76	1839.08	701.75
300B	1889.72	2857.14	1875.04	869.57
400B	2176.87	2560.00	1882.35	862.53
576B	2215.42	2776.00	2183.91	901.76
1536B	2631.35	3294.32	2262.91	1013.87
4096B	2810.39	3592.96	2383.19	1077.90
> 4096B (1MiB)	2836.61	3672.38	2465.53	1102.85

Performance on Hygon C86-4G. The third testing environment consisted of a Hygon C86-4G processor running Kylin Linux Advanced Server V10, utilizing g++ 7.3.0 and OpenSSL 1.1.1f. The benchmark was pinned to Core 32 on NUMA Node 2. Table 12 presents the speed test results for various message lengths. Compared to SHA3-512, Vedak-512 is faster.

Table 12: Software performance of Vedak on Hygon C86-4G (cycle/Byte)

Data Level	Message length	SHA3-512	Vedak-512	Vedak-768	Vedak-1024
S1	32B	50.625	75.938	78.188	140.906
S2	128B	20.766	18.984	39.023	73.828
S3	512B	17.057	14.467	24.276	46.986
S4	1024B	15.407	14.476	21.815	42.530
S5	4096B	14.132	13.274	19.984	39.036
S6	8192B	14.052	12.970	19.671	38.525
S7	16384B	14.063	12.973	19.527	38.204
S8	65536B	14.003	12.892	19.405	38.041

5.2 Implementation Performance using AVX-512 Instruction Set

The software implementation performance of **Vedak** was tested on a 11th Gen Intel(R) Core(TM) i7-11850H @ 2.50 GHz platform (16 GB memory, Windows 10 64-bit) supporting the AVX-512 instruction set, with the compilation environment being Microsoft Visual Studio 2022. The performance metrics are presented in cycles per byte (cpb). The test results are shown in Table 13.

Table 13: Software performance of Vedak using AVX-512 instruction set

Variant	Message length	Throughput (Mbps)	Speed (cycle/Byte)
Vedak-512	8B	145.76	136.998
Vedak-512	32B	491.30	40.643
Vedak-512	64B	1035.36	19.286
Vedak-512	256B	2007.78	9.945
Vedak-512	1024B	2990.91	6.676
Vedak-512	4096B	3083.68	6.475
Vedak-512	1MiB	3122.00	6.396
Vedak-768	8B	115.64	172.682
Vedak-768	32B	498.92	40.023
Vedak-768	64B	997.72	20.014
Vedak-768	256B	1499.81	13.314
Vedak-768	1024B	2018.94	9.890
Vedak-768	4096B	2289.13	8.723
Vedak-768	1MiB	2314.03	8.629
Vedak-1024	8B	60.88	328.011
Vedak-1024	32B	257.24	77.625
Vedak-1024	64B	365.14	54.686
Vedak-1024	256B	696.20	28.682
Vedak-1024	1024B	1040.24	19.196
Vedak-1024	4096B	1131.66	17.645
Vedak-1024	1MiB	1167.98	17.096

Under the same environment, the optimized implementation provided by OpenSSL was used to test the processing speed of long messages for **SHA3-512**, achieving a performance of 9.44 cycles per byte. For comparison, **Vedak-512** achieves 6.396 cycles/byte in the long message, which corresponds to an improvement of approximately 47.6% over SHA3-512.

6 Features

The design features of Vedak can be summarized in the following three aspects.

1. **A large-state cryptographic permutation well suited for SIMD instruction sets.** Vedak-p uses nibbles as its basic processing units and adopts operations such as 4-bit S-boxes, nibble slicing, and shuffle operations. Its design fully takes into account the capabilities of AVX2, AVX512, and NEON instruction sets, leading to better software performance than SHA3-512 in the reported benchmarks. Moreover, the modular structure, lightweight low-latency S-boxes, and linear transformations make the hardware implementation of the Vedak algorithm simple and efficient.
2. **A security evaluation framework for large-state primitives.** To address the difficulty of precisely characterizing complex large-state linear layers and the limited solving efficiency in automated differential analysis, we propose a generative constraint modeling technique. The nibble slicing of Vedak adopts optimal binary-field matrices. Combined with the design principles of S-boxes and branch numbers, Vedak provides provable security conclusions against typical cryptanalytic methods.
3. **Good scalability.** Vedak adopts the sponge construction and uses a large-state cryptographic permutation to support high-security-strength claims. It can be easily extended to support multiple digest lengths and is suitable for applications requiring different security strengths. Meanwhile, Vedak inherits the advantages and mature security-analysis results of the sponge construction. It resists length-extension attacks and enjoys provable security guarantees in both classical and quantum settings.

7 Statistical Tests

The statistical tests for Vedak-512, Vedak-768, and Vedak-1024 are conducted using seven message generation methods: random message pattern, counter mode, output feedback mode, low Hamming weight mode, high Hamming weight mode, diffusion test samples, and confusion test samples. The significance level is set to 0.010, the message length is 10^6 bits, and the number of test sequences is 1000.

The test items include monobit frequency, frequency within a block, poker test, overlapping template test, total number of runs, runs distribution, binary derivative, autocorrelation, matrix rank, cumulative sums in the forward direction, cumulative sums in the backward direction, approximate entropy, linear complexity, universal statistical test, and discrete Fourier transform.

7.1 Statistical Randomness Test Results of Vedak-512

The summarized results of all test items for Vedak-512 are shown in Table 14.

Table 14: Randomness test results of Vedak-512

Test Item	Percentage of Passed Tests (%)					
	Random Message Mode	Counter Mode	Output Feedback Mode	Low Hamming Weight Mode	High Hamming Weight Mode	Diffusion Test Samples
Monobit Frequency	98.80	99.10	98.50	98.50	99.50	98.80
Frequency within a Block (100)	98.90	99.30	99.30	99.10	99.00	98.90
Poker (4)	99.10	98.90	98.40	99.60	99.30	99.10
Overlapping Template (5)-1	99.00	99.50	98.90	98.80	99.10	98.90
Overlapping Template (5)-2	99.10	99.20	98.80	98.90	99.20	99.10
Total Number of Runs	99.00	99.20	98.90	98.90	98.20	98.70
Runs Distribution	98.40	99.20	98.80	98.50	99.20	99.10
Binary Derivative (7)	99.20	99.10	98.80	98.80	99.10	98.70
Autocorrelation (2)	99.00	98.90	99.10	99.50	99.10	98.80
Matrix Rank	98.10	99.10	99.10	99.20	99.50	98.90
Cumulative Sums (Forward)	98.80	99.10	98.40	98.40	99.50	98.80
Cumulative Sums (Backward)	98.90	99.00	98.50	99.00	99.40	98.40
Approximate Entropy (5)	99.20	99.10	99.00	99.10	99.20	98.30
Linear Complexity (500)	99.10	99.00	98.90	98.50	98.70	99.20
Universal Statistical Test (7)(1280)	98.70	99.40	98.80	98.80	98.90	99.10
Discrete Fourier Transform	99.20	98.70	99.20	99.20	98.80	99.20

7.2 Statistical Randomness Test Results of Vedak-768

The summarized results of all test items for Vedak-768 are shown in Table 15.

Table 15: Randomness test results of Vedak-768

Test Item	Percentage of Passed Tests (%)					
	Random Message Mode	Output Feedback Mode	Low Hamming Weight Mode	High Hamming Weight Mode	Diffusion Test Samples	Confusion Test Samples
Monobit Frequency	99.20	98.90	99.20	98.70	98.60	98.90
Frequency within a Block (100)	99.10	98.60	99.20	98.80	99.50	99.20
Poker (4)	99.10	99.30	99.10	99.40	98.80	99.50
Overlapping Template (5)-1	99.10	98.90	98.70	99.10	98.50	99.00
Overlapping Template (5)-2	99.60	98.20	98.60	99.00	98.80	99.10
Total Number of Runs	99.00	99.00	98.10	99.00	98.60	98.80
Runs Distribution	98.70	99.70	98.70	99.10	98.30	98.70
Binary Derivative (7)	98.50	99.10	98.80	99.00	99.40	98.90
Autocorrelation (2)	99.10	98.40	99.20	99.10	98.90	99.40
Matrix Rank	98.70	98.70	98.90	99.00	99.30	99.10
Cumulative Sums (Forward)	99.40	98.70	99.20	98.80	98.70	99.20
Cumulative Sums (Backward)	98.90	98.60	99.00	98.60	98.90	99.00
Approximate Entropy (5)	99.40	98.70	98.90	99.50	99.30	99.30
Linear Complexity (500)	98.90	99.10	98.90	99.20	99.00	98.80
Universal Statistical Test (7)(1280)	98.90	98.90	98.50	98.60	99.20	98.40
Discrete Fourier Transform	98.90	99.50	99.20	98.40	98.70	99.30

7.3 Statistical Randomness Test Results of Vedak-1024

The summarized results of all test items for Vedak-1024 are shown in Table 16.

8 Instance

Several Vedak test vectors are provided below to validate implementations. All messages and digests in the test vectors are written in hexadecimal. For non-byte-aligned input messages, only the first bits specified by `Msg_Len` are involved in the computation.

8.1 Vedak-512

Example 1:

`Msg_Len = 0`

Table 16: Randomness test results of Vedak-1024

Test Item	Percentage of Passed Tests (%)					
	Random Message Mode	Output Feedback Mode	Low Hamming Weight Mode	High Hamming Weight Mode	Diffusion Test Samples	Confusion Test Samples
Monobit Frequency	99.10	99.10	98.90	99.30	98.50	99.10
Frequency within a Block (100)	99.40	99.60	99.10	99.50	99.20	99.30
Poker (4)	98.90	98.90	98.30	99.20	98.80	99.20
Overlapping Template (5)-1	99.00	99.20	99.10	99.20	98.80	99.30
Overlapping Template (5)-2	99.30	99.10	99.30	99.30	99.00	99.40
Total Number of Runs	98.90	99.00	98.80	99.20	98.90	99.10
Runs Distribution	98.90	99.20	99.10	99.10	98.70	98.60
Binary Derivative (7)	98.70	99.10	98.70	98.90	99.20	99.20
Autocorrelation (2)	99.00	99.10	99.20	99.00	98.80	98.70
Matrix Rank	98.70	98.90	99.20	99.30	99.20	99.50
Cumulative Sums (Forward)	99.10	99.20	98.90	99.30	98.20	98.80
Cumulative Sums (Backward)	99.20	98.90	99.10	99.40	98.60	99.50
Approximate Entropy (5)	98.90	99.30	98.90	99.10	98.60	99.20
Linear Complexity (500)	99.10	98.90	99.00	99.40	99.30	98.70
Universal Statistical Test (7)(1280)	98.40	98.70	99.10	98.60	98.60	98.70
Discrete Fourier Transform	98.70	99.10	99.20	98.40	99.00	99.00

Msg =
Dst_Len = 512
Dst = 3C1C2861280A70892B6719B7D338AB329C0B4B45C6B46B6B47950665ED5827FF9B68
21925DE2FB7FDCD8F3F3E4013CF08437C3A3C2720404F03EE8A8EA8BAF39

Example 2:

Msg_Len = 8388608 (2^{23})
Msg_Seed = 4B41545F325F32334B41545F325F32334B41545F325F32334B41545F325F32334B
41545F325F32334B41545F325F32334B41545F325F32334B41545F325F3233
Msg_Exp = B6E2C825BE0E7D79 70B7D148BC6FCCEC
Dst_Len = 512
Dst = FE1EC867D66164C06A0A00FABFD5B547C944F0CEDB9EAB9CEFA0B8EBF230E7A
A5C8EC36341D20A43A5C8A21E0D6FF6AFA6BE4292C99CF78413D6A9B6AFACCFB1

Example 3:

Msg_Len = 8589934592 (2^{23})
Msg_Seed = 4B41545F325F33334B41545F325F33334B41545F325F33334B41545F325F33334B
41545F325F33334B41545F325F33334B41545F325F33334B41545F325F3333
Msg_Exp = AE6416F9B6D4C97A 31B80BC6E737A914
Dst_Len = 512
Dst = 05562D79644793D27722D8D0CD207F1B77B63B57BC55B7702E645FF48A04FC455C99
F08EB4757E60B5761874BB5718ED191A3F34F281D7149A9D86B7E87C9E6E

Example 4:

Msg_Len = 8192
Msg = 2D911820A8ED087902CCFEEAD2C91785CEA0289755C29F651DB41AF1D8434774FC
A391F2F7F9473A2619DC75FCC817E42B7A591AF2E95E3D43EF7D4EA978EEABF0F2E699
7B64CE29E776A297B5A8EDBC10C7804C2EBF0F76DBBD78EBEFAB289629B5A9BBA6B2
F0F87D59D73462F3076F35B6C24B47A2B5621A7C68274CE1E54F0388BE2FB29051FC9E3B7
E9E137D4CAB088849D39ACBF9D919F0CC38F95133345E329F79C7E12D1B3ADFB2F62A42
CF16D2489BCB8351016AB5C2F896397BED83BF9749FB9E7F6AA0B13B16D0A6242D9AA2
07B0AA23F0457C8DCAC96014727ABD5193AA95E65CBF2021D8BCE16DB4ECC701EB3F7
34A4AAD55F6F8D6BD47B9E426443C557AF8EF48ABE5E9995FBA06E0B13F116F4CBB11B
1E82EFFF5B346D85DF49A89D515D4BFE2C901E41CFDDF463AA5C5E2354E934E447FBB
C28FFB31FA2C3F8A8056E4DF8E51FF22C88EA9F78ED0633A2EDD5095AF33FC685916DB
679C9A1DF1D2EAC1A5F8200093F1CCCE807B1D46BEECE6CECEF7BF74F39EF21A332D

4EB75B96953387CA485D3C428C392B86928D565886525F9AF84A0283F05981813C7300DA10
3F39B5F9C3DDFCFA401AD90C5759FDCB56EA2597D36114A2A3DE0BB44FD5472C712EF
A1188C6D60C4FCCD3BECCF1887DAD096BAC54689DED755ED453A4C1CE0CA981288C2
EA0D7EB4E71C53C250DF0C876CFBC45F6AB2821EC8FE0ED5AC5293E644C4931FD03E07
02FB3478E1FFC39A96D4C06035D30B72F3A5FE6233945A3A515FB55D419DF2EFBA5E358
C65E33FE54C158B6215363BBD338A9EC3307821865FCE5BFCFBF4B4F1A8C5FBD446469A
1A65CF45973BDFE3C4A26F5EDFECDF226E50F2A11A664CA45B8E20BA412ECE8A5FEE3
23B69F5835EEAA3BB95DDE9F11291E7473F46AE99A9F0413F855D05EBD57E597D45DC50
B792E7881E51D84C77EDC8016F6864971A0EA72876761A8E523F1AFB1254DD2A399BE12D
C6B0FA38174D292990114924468B6D86845A20701EAC464AC9887702C0B5B130288D08F93A
F671E66DAAD02D64B60C61CE47066BC7AFEDA951B3170798F357E4E28D3CC123B29621A
B500FE112C05D2BC66A98AB42FF129A19A6D39413D3F58672D3EB6C0ADEC923F2A2DB9
E8D847B53929D32397D380AC1A4835CB11E632C4CA5B73ED02885717116B8EB2010CED40
4F9CA517D41D7F25AB2EC0F3FF4D81EEF0C90602E284AC86EE466A2E18525BEFDE2292
83599A2161308A685C0E35A26C480BAA36C87250077E93D75B4B6930D3C5A9CDFB8E08652
BFE4DB82CC50D6D37AE3B83BFB63C3A4D8AC4CF5AF7BE1DA11EABCEB3C20B165176
50CB28E5CE5E23AD22DF395A7FEE3FCCC1540FED322954AE ECBAC6C3F0F94099A5328
47B9E4F888A37716FDCE660CC13122AC974D314E4A924BDB550FFC029E36FAC6D91E7A8
07DBC461377D

Dst_Len = 512

Dst = 0A0056117218525BFA1775C65AF1A8F033BAE081FB441A5BE5118906D0FAA8E37B7
54E7523DD76A543092F66990D63BBFDF5597FB70B0E7A9D13153EDEA80E62

8.2 Vedak-768

Example 1:

Msg_Len = 0

Msg =

Dst_Len = 768

Dst = 2D535433095A3EB2AE6CEC3C3BE08D69FE1C5436E392A45A2B888BFDDAB2AB6F
9F6702EA9E68A0AB16A04E3F42A986B96AEC8DF56D9B6B1892CFA3A5341D6D30C071FB
A5E309E1EA0B6D9CD1CF3BFFC733B87ECBCB4489021004D29F13124611

Example 2:

Msg_Len = 8388608 (2^{23})

Msg_Seed = 4B41545F325F32334B41545F325F32334B41545F325F32334B41545F325F32334B

41545F325F32334B41545F325F32334B41545F325F32334B41545F325F3233
Msg_Exp = B6E2C825BE0E7D79 70B7D148BC6FCCEC
Dst_Len = 768
Dst = B6942D9E1A6613645A92E28DD6A727E08440CDBCBBFF48C74F09E0377854BD08D59
D4EB4C392AA459AA08EC4869F4613D0FCC8EB01B631F992009DB0496BBBA5FA21BDF52
CC3E3A585B058CC2E9139D0A30369F189676D9FF186B416865CF0F86

Example 3:

Msg_Len = 8589934592 (2^{33})
Msg_Seed = 4B41545F325F33334B41545F325F33334B41545F325F33334B41545F325F33334B
41545F325F33334B41545F325F33334B41545F325F33334B41545F325F3333
Msg_Exp = AE6416F9B6D4C97A 31B80BC6E737A914
Dst_Len = 768
Dst = 51E4BCF6BB11742259B171F3F6EBD1573C0CE0EC407007AC044632CE61EDA8B790
352995AC887994A6C1510ABEB39B45DEE1D1DEEFBF3A919AF1BBD7E93D4460FBFA490
F1B5F07976B7DFA7B9C31001F8BE37E634737FE1A38542C60E5D81CC6

Example 4:

Msg_Len = 8192
Msg = 2D911820A8ED087902CCFEEAD2C91785CEA0289755C29F651DB41AF1D8434774FC
A391F2F7F9473A2619DC75FCC817E42B7A591AF2E95E3D43EF7D4EA978EEABF0F2E699
7B64CE29E776A297B5A8EDBC10C7804C2EBF0F76DBBD78EBEFAB289629B5A9BBA6B2
F0F87D59D73462F3076F35B6C24B47A2B5621A7C68274CE1E54F0388BE2FB29051FC9E3B7
E9E137D4CAB088849D39ACBF9D919F0CC38F95133345E329F79C7E12D1B3ADFB2F62A42
CF16D2489BCB8351016AB5C2F896397BED83BF9749FB9E7F6AA0B13B16D0A6242D9AA2
07B0AA23F0457C8DCAC96014727ABD5193AA95E65CBF2021D8BCE16DB4ECC701EB3F7
34A4AAD55F6F8D6BD47B9E426443C557AF8EF48ABE5E9995FBA06E0B13F116F4CBB11B
1E82EFFF5B346D85DF49A89D515D4BFE2C901E41CFDDF463AA5C5E2354E934E447FBB
C28FFB31FA2C3F8A8056E4DF8E51FF22C88EA9F78ED0633A2EDD5095AF33FC685916DB
679C9A1DF1D2EAC1A5F8200093F1CCCE807B1D46BEECE6CECEF7BF74F39EF21A332D
4EB75B96953387CA485D3C428C392B86928D565886525F9AF84A0283F05981813C7300DA10
3F39B5F9C3DDFCFA401AD90C5759FDCB56EA2597D36114A2A3DE0BB44FD5472C712EF
A1188C6D60C4FCCD3BECCF1887DAD096BAC54689DED755ED453A4C1CE0CA981288C2
EA0D7EB4E71C53C250DF0C876CFBC45F6AB2821EC8FE0ED5AC5293E644C4931FD03E07
02FB3478E1FFC39A96D4C06035D30B72F3A5FE6233945A3A515FB55D419DF2EFBA5E358
C65E33FE54C158B6215363BBD338A9EC3307821865FCE5BFCFBF4B4F1A8C5FBD446469A

1A65CF45973BDFE3C4A26F5EDFECDF226E50F2A11A664CA45B8E20BA412ECE8A5FEE3
23B69F5835EEAA3BB95DDE9F11291E7473F46AE99A9F0413F855D05EBD57E597D45DC50
B792E7881E51D84C77EDC8016F6864971A0EA72876761A8E523F1AFB1254DD2A399BE12D
C6B0FA38174D292990114924468B6D86845A20701EAC464AC9887702C0B5B130288D08F93A
F671E66DAAD02D64B60C61CE47066BC7AFEDA951B3170798F357E4E28D3CC123B29621A
B500FE112C05D2BC66A98AB42FF129A19A6D39413D3F58672D3EB6C0ADEC923F2A2DB9
E8D847B53929D32397D380AC1A4835CB11E632C4CA5B73ED02885717116B8EB2010CED40
4F9CA517D41D7F25AB2EC0F3FF4D81EEF0C90602E284AC86EE466A2E18525BEFDE2292
83599A2161308A685C0E35A26C480BAA36C87250077E93D75B4B6930D3C5A9CDFB8E08652
BFE4DB82CC50D6D37AE3B83BFB63C3A4D8AC4CF5AF7BE1DA11EABCEB3C20B165176
50CB28E5CE5E23AD22DF395A7FEE3FCCC1540FED322954AE ECBAC6C3F0F94099A5328
47B9E4F888A37716FDCE660CC13122AC974D314E4A924BDB550FFC029E36FAC6D91E7A8
07DBC461377D

Dst_Len = 768

Dst = CC1A5770B0180FA8B9AEAA49551B237A9562BC80F04D09E5C2366AA2948FF73B7B
1A8C021273B9AB250D104995E1D52C6E091B300B62D99F2016C6273BDE27FE2403DFDAE5
168C0C1E1AAA55C363EA840F064DA4A3D34862706C3FEE09F47937

8.3 Vedak-1024

Example 1:

Msg_Len = 0

Msg =

Dst_Len = 1024

Dst = 71ABEDCA7503D3D6B64A08F9794AE32F4732BC71056EDA9CF0F6912E5595A76536
A414EF078B829CA8B37D13A8834FDDC7B189DDF019812E2E6B2765FA562E19120FF59358
5FBDCF2F4DB33335214EB37C973D8A33218B64578D03AB58FB9F1DD082EB1DCDBD8151
2F177BC7CFD3924AFB3BC0D6DB75BF751D8C24E897BADCAA

Example 2:

Msg_Len = 8388608 (2^{23})

Msg_Seed = 4B41545F325F32334B41545F325F32334B41545F325F32334B41545F325F32334B
41545F325F32334B41545F325F32334B41545F325F32334B41545F325F3233

Msg_Exp = B6E2C825BE0E7D79 70B7D148BC6FCCEC

Dst_Len = 1024

Dst = D45899F42B6DC0C6B3AFFB31B2E7DCEE78E6BA840C86BEDA069B424E518831AA

F44ACE6E95012C1D0C3ECBB21935BF0E2F21515F025A5AD5DA8141F5D11CB63B41E73E
1E3B986DE086870A5622C0EAE032A2F4717BDD64CF63C54BC086B68F967F1787901E78120
DF76FB7C6656AEF024E14F801AA3E51FD4702155DFA7757BA

Example 3:

Msg_Len = 8589934592 (2^{33})

Msg_Seed = 4B41545F325F33334B41545F325F33334B41545F325F33334B41545F325F33334B
41545F325F33334B41545F325F33334B41545F325F33334B41545F325F3333

Msg_Exp = AE6416F9B6D4C97A 31B80BC6E737A914

Dst_Len = 1024

Dst = 1E712E974AFEFCC670DE1091FE7B86BD107FCA2D2E077F1E19050397715702893FB
166242393A6A6309AE1DFD24682793EDE8DED171C3566037F8D246A40FCC3ED4A33DF13
446C5193700BC816DF61007423E554E8BC1E62EF6FB07352E97474827D60805BD4D6D79808
642F38DDE58E75EE30845E0C65B56E7061A01F10EED1

Example 4:

Msg_Len = 8192

Msg = 2D911820A8ED087902CCFEEAD2C91785CEA0289755C29F651DB41AF1D8434774FC
A391F2F7F9473A2619DC75FCC817E42B7A591AF2E95E3D43EF7D4EA978EEABF0F2E699
7B64CE29E776A297B5A8EDBC10C7804C2EBF0F76DBBD78EBEFAB289629B5A9BBA6B2
F0F87D59D73462F3076F35B6C24B47A2B5621A7C68274CE1E54F0388BE2FB29051FC9E3B7
E9E137D4CAB088849D39ACBF9D919F0CC38F95133345E329F79C7E12D1B3ADFB2F62A42
CF16D2489BCB8351016AB5C2F896397BED83BF9749FB9E7F6AA0B13B16D0A6242D9AA2
07B0AA23F0457C8DCAC96014727ABD5193AA95E65CBF2021D8BCE16DB4ECC701EB3F7
34A4AAD55F6F8D6BD47B9E426443C557AF8EF48ABE5E9995FBA06E0B13F116F4CBB11B
1E82EFFF5B346D85DF49A89D515D4BFE2C901E41CFDDF463AA5C5E2354E934E447FBB
C28FFB31FA2C3F8A8056E4DF8E51FF22C88EA9F78ED0633A2EDD5095AF33FC685916DB
679C9A1DF1D2EAC1A5F8200093F1CCCE807B1D46BEECE6CECEF7BF74F39EF21A332D
4EB75B96953387CA485D3C428C392B86928D565886525F9AF84A0283F05981813C7300DA10
3F39B5F9C3DDFCFA401AD90C5759FDCB56EA2597D36114A2A3DE0BB44FD5472C712EF
A1188C6D60C4FCCD3BECCF1887DAD096BAC54689DED755ED453A4C1CE0CA981288C2
EA0D7EB4E71C53C250DF0C876CFBC45F6AB2821EC8FE0ED5AC5293E644C4931FD03E07
02FB3478E1FFC39A96D4C06035D30B72F3A5FE6233945A3A515FB55D419DF2EFBA5E358
C65E33FE54C158B6215363BBD338A9EC3307821865FCE5BFCFBF4B4F1A8C5FBD446469A
1A65CF45973BDFE3C4A26F5EDFECDF226E50F2A11A664CA45B8E20BA412ECE8A5FEE3
23B69F5835EEAA3BB95DDE9F11291E7473F46AE99A9F0413F855D05EBD57E597D45DC50

B792E7881E51D84C77EDC8016F6864971A0EA72876761A8E523F1AFB1254DD2A399BE12D
C6B0FA38174D292990114924468B6D86845A20701EAC464AC9887702C0B5B130288D08F93A
F671E66DAAD02D64B60C61CE47066BC7AFEDA951B3170798F357E4E28D3CC123B29621A
B500FE112C05D2BC66A98AB42FF129A19A6D39413D3F58672D3EB6C0ADEC923F2A2DB9
E8D847B53929D32397D380AC1A4835CB11E632C4CA5B73ED02885717116B8EB2010CED40
4F9CA517D41D7F25AB2EC0F3FF4D81EEF0C90602E284AC86EE466A2E18525BEFDE2292
83599A2161308A685C0E35A26C480BAA36C87250077E93D75B4B6930D3C5A9CDFB8E08652
BFE4DB82CC50D6D37AE3B83BFB63C3A4D8AC4CF5AF7BE1DA11EABCEB3C20B165176
50CB28E5CE5E23AD22DF395A7FEE3FCCC1540FED322954AE ECBAC6C3F0F94099A5328
47B9E4F888A37716FDCE660CC13122AC974D314E4A924BDB550FFC029E36FAC6D91E7A8
07DBC461377D

Dst_Len = 1024

Dst = E59874C5B0098AB3F5B2C34CC8551500F3CDAEFD3C5178D16B5739DE584046DAE6
7949BC4AC0B340E7D0926FBCFADB42D609952F8EE3E75D191FC8A66AA6DDAF45F365E
3A931BE16BCD09F2A674857B05C389FE612F3160EFD92416B9DA83539AA430BA9E8D2706
C75C133A6C7AED6C80A31C361603861794E18803E32A7DE73

References

- [1] Bertoni G., Daemen J., Peeters M., Van Assche G.: Cryptographic Sponge Functions. (2011).
- [2] Ruichen Wu and Lei Zhang. Construction of low-latency S-boxes and the study of bidirectional low-latency properties[J]. Acta Electronica Sinica, 2024(11).
- [3] Guo, Z., Liu, R., Gao, S., Wu, W., Lin, D.: Direct Construction of Optimal Rotational-XOR Diffusion Primitives. IACR Transactions on Symmetric Cryptology, 2017(4), 169–187 (2017).
- [4] Alfarano, G. N., Beierle, C., Isobe, T., K”olbl, S., Leander, G.: ShiftRows Alternatives for AES-like Ciphers and Optimal Cell Permutations for Midori and Skinny. IACR Transactions on Symmetric Cryptology, 2018(2), 20–47 (2018).
- [5] Wenling Wu, Lei Zhang, Yafei Zheng, and Lingchen Li. The block cipher uBlock. Journal of Cryptologic Research, 2019, 6(6): 690–703.
- [6] Bertoni, G., Daemen, J., Peeters, M., Van Assche, G.: On the Indifferentiability of the Sponge Construction. In: Smart, N. (ed.) EUROCRYPT 2008. Lecture Notes in Computer Science, vol. 4965, pp. 181–197. Springer, Heidelberg (2008).

- [7] National Institute of Standards and Technology, *FIPS 202: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*, 2015.
- [8] Lefevre, C., Mennink, B.: Tight Preimage Resistance of the Sponge Construction. Cryptology ePrint Archive, Paper 2022/734 (2022).
- [9] Sun, S., Li, S., Zhang, Z., Lefevre, C., Mennink, B., Qin, Z., Feng, D.: Permutation-Based Hashing with Stronger (Second) Preimage Resistance: Application to Hash-Based Signature Schemes. Cryptology ePrint Archive, Paper 2025/963 (2025).
- [10] Alagic, G., Carolan, J., Majenz, C., Tokat, S.: The Sponge is Quantum Indifferentiable. Cryptology ePrint Archive, Paper 2025/731 (2025).
- [11] Grover, L. K.: A Fast Quantum Mechanical Algorithm for Database Search. In: Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC 1996), pp. 212–219 (1996).
- [12] Ambainis, A.: Polynomial Degree and Lower Bounds in Quantum Complexity: Collision and Element Distinctness with Small Range. Theory of Computing, 1(1), 37–46 (2005).
- [13] Brassard, G., Høyer, P., Tapp, A.: Quantum Algorithm for the Collision Problem. ACM SIGACT News, 28(2), 14–19 (1997).
- [14] Biham, E., Shamir, A.: Differential Cryptanalysis of DES-like Cryptosystems. Journal of Cryptology, 4(1), 3–72 (1991).
- [15] Daemen, J., Rijmen, V.: The Wide Trail Design Strategy. In: Honary, B. (ed.) Cryptography and Coding 2001. Lecture Notes in Computer Science, vol. 2260, pp. 222–238. Springer, Heidelberg (2001).
- [16] Mouha, N., Wang, Q., Gu, D., Preneel, B.: Differential and Linear Cryptanalysis Using Mixed-Integer Linear Programming. In: Information Security and Cryptology – INSCRYPT 2011. LNCS 7537, pp. 57–76. Springer, Heidelberg (2012).
- [17] Zhang, Y., Wu, W., Zhang, L., Zheng, Y.: Differential cryptanalysis based on generative constraints and meet-in-the-middle techniques[J]. Journal of Cryptologic Research (Chinese and English), 2026 (accepted).
- [18] Matsui, M.: Linear Cryptanalysis Method for DES Cipher. In: Advances in Cryptology – EUROCRYPT’93. LNCS 765, pp. 386–397. Springer, Heidelberg (1994).
- [19] Todo, Y., Morii, M.: Bit-Based Division Property and Application to Simon Family. In: Fast Software Encryption – FSE 2016. LNCS 9783, pp. 357–377. Springer, Heidelberg (2016).

- [20] Qin, L., Hua, J., Dong, X., Yan, H., Wang, X.: Meet-in-the-Middle Preimage Attacks on Sponge-Based Hashing. In: *Advances in Cryptology – EUROCRYPT 2023*. Lecture Notes in Computer Science, vol. 14006, pp. 158–188. Springer, Cham (2023).
- [21] Bao, Z., Guo, J., Wang, M., Liu, Y., Zhang, H.: Automatic Search of Meet-in-the-Middle Preimage Attacks on AES-Like Hashing. In: *Advances in Cryptology – CRYPTO 2021, Part III*. Lecture Notes in Computer Science, vol. 12827, pp. 484–514. Springer, Cham (2021).
- [22] Sun S, Hu L, Wang P, et al. Automatic security evaluation and (related-key) differential characteristic search: Application to SIMON, PRESENT, LBlock, DES(L) and other bit-oriented block ciphers [C/OL]//Sarkar P, Iwata T. *Lecture Notes in Computer Science: volume 8873 Advances in Cryptology - ASIACRYPT 2014, Part I*. Springer, 2014: 158-178.
- [23] Todo, Y, Sasaki, Y: Designing S-Boxes Providing Stronger Security Against Differential Cryptanalysis for Ciphers Using Byte-Wise XOR. *SAC 2021*: 179-199
- [24] Qin, L., Hua, J., Dong, X., Yan, H., Wang, X. (2023). Meet-in-the-Middle Preimage Attacks on Sponge-Based Hashing. In: Hazay, C., Stam, M. (eds) *Advances in Cryptology – EUROCRYPT 2023*. EUROCRYPT 2023. Lecture Notes in Computer Science, vol 14007. Springer, Cham.
- [25] Bao, Z. et al. (2021). Automatic Search of Meet-in-the-Middle Preimage Attacks on AES-like Hashing. In: Canteaut, A., Standaert, FX. (eds) *Advances in Cryptology – EUROCRYPT 2021*. EUROCRYPT 2021. Lecture Notes in Computer Science, vol 12696. Springer, Cham.
- [26] Zhang Y, Wu W, Zhang L, et al., Improved modeling methods for automated differential analysis of block ciphers: applications to ublock-256 and PRESENT, *Chinese Journal of Electronics*, vol. 35, no. 3, pp. 1–16, 2026. DOI: 10.23919/cje.2025.00.032